

AMS/WMS Platform

Combined Technical Documentation

Backend Business Service and Frontend Application

Generated 2026-08-27

Contents

Part I — Backend Business Service	3
1. Scope	3
2. System context	3
3. Stack	3
4. Source architecture	3
5. Lifecycle	4
6. Module status	4
7. API catalog	4
Operations and shared	4
Receiving and returns	4
Gate, dock, receiving, and exit	5
Storage	5
Procurement	5
8. Core workflows	5
9. Data and migrations	6
10. Authentication and authorization	6
11. Configuration	6
12. Local development	6
13. Tests and observability	7
14. Extension rules	7
15. Known risks and production checklist	7
Part II — Frontend Application	8
1. Scope	8
2. Stack	8
3. Runtime and source architecture	8
4. Roles and navigation	8
5. Route catalog	9
Shared	9
Gate and warehouse	9
Procurement, supplier, and finance	9
6. Backend integration	9
7. Authentication	10
8. State, UI, and errors	10
9. Development and verification	10
10. Adding a feature	11
11. Known risks and production checklist	11

Part I — Backend Business Service

1. Scope

`business-service` is the Python/FastAPI backend for AMS/WMS operations. It owns procurement, supplier management, gate entry, inbound receiving, storage/putaway, returns, notifications, dashboards, and transactional events. Identity is intended to come from the separate Java `auth-service`; local modes also provide development identities.

This document describes the code currently in this repository. FastAPI's `/docs` and `/openapi.json` are the definitive field-level API contracts.

2. System context

```

React frontend (:8080)
| HTTP + Bearer JWT
v
FastAPI business-service (:8080) ---- JWKS ----> Java auth-service (:8080)
|---- async SQLAlchemy -----> PostgreSQL business DB (:5433)
|---- transactional outbox -----> Kafka (:9092)
|<--- notification consumer -----|
+---- media files, SMTP, Gemini/OCR (when configured)

```

The parent `docker-compose.yml` defines PostgreSQL, Kafka, Redis, auth, and business services. Redis is configured but is not a primary persistence mechanism in current workflows.

3. Stack

Concern	Technology
Runtime/API	Python 3.12–3.13, FastAPI 0.115, Uvicorn, Pydantic 2
Data	PostgreSQL, SQLAlchemy 2 async, asyncpg, Alembic
Messaging/jobs	Kafka/aiokafka, APScheduler
Security	Bearer JWT, RS256/JWKS, python-jose
Observability	structured logging, request IDs, Prometheus, optional OpenTelemetry
Files/docs/vision	multipart, ReportLab, Pillow, Gemini, OpenCV/Tesseract adapters
Tests	pytest, pytest-asyncio, HTTPX

Pinned packages are in `pyproject.toml` and `requirements.txt`.

4. Source architecture

```

business-service/
├─ app/
│  ├─ main.py app factory, lifespan, router registration
│  ├─ config/ environment settings
│  ├─ database/ async engine/session and base
│  ├─ security/ JWT and authorization dependencies
│  ├─ middleware/ request context and error translation
│  ├─ common/domain/ aggregate, event, exception foundations
│  ├─ events/ outbox model/repository
│  ├─ kafka/ producer/consumer wrappers
│  ├─ workers/ outbox and notification workers
│  └─ modules/ bounded business modules
├─ alembic/ schema migrations
├─ tests/ unit and integration tests
├─ scripts/ administrative utilities
└─ .env.example

```

The intended dependency direction is `infrastructure/API -> application/use cases -> domain`; persistence implementations satisfy application repository protocols. Receiving, returns, notification, and master data follow this separation closely. The active gate and procurement routers also contain significant orchestration and some direct persistence/reporting logic.

5. Lifecycle

`app.main:create_app()` installs request-context and CORS middleware, exception handlers, routers, static media, health checks, and optional Prometheus metrics.

Startup performs compatibility DDL, attempts to start Kafka, schedules the transactional-outbox relay, schedules an upcoming-arrival check every 60 seconds, and starts the notification consumer. Shutdown stops the scheduler, consumer, and producer. Kafka errors are handled defensively so local HTTP development can continue.

The startup `CREATE/ALTER TABLE IF NOT EXISTS` statements are legacy compatibility behavior, not a migration strategy. New schema work belongs in Alembic.

6. Module status

Module	Status and responsibility
procurement	Active: material requests/stock, suppliers, RFQs, quotations, selection, POs, finance approval, ASNs, notifications, supplier login, search, PDFs/uploads
gate	Active: OCR/ANPR, entry/verification, docks, receiving checks, handling units, exit, GRN posting, inventory transactions
storage	Active: putaway tasks, locations, handling-unit lookup, task execution and balances
receiving	Active: domain-centric GRN confirmation and retrieval
returns	Active: return creation and retrieval
notification	Active: event webhook and Kafka-consumed notification log
masterdata	Implemented but its separate <code>/api/v1/suppliers</code> router is not mounted; UI uses procurement supplier routes
approval, assembly, dispatch, disposition, erp, quality, shipment, traceability	Skeletons; no mounted API

Older migration notes and the original README predate parts of the active gate, procurement, and storage implementation.

7. API catalog

Operations and shared

Method/path	Purpose
GET <code>/health</code>	liveness
GET <code>/health/ready</code>	static readiness marker
GET <code>/metrics</code>	Prometheus metrics when enabled
GET <code>/docs</code> , GET <code>/openapi.json</code>	API documentation/schema
GET <code>/media/...</code>	process-local uploaded media
POST <code>/webhooks/events</code>	notification/integration-event ingestion

Receiving and returns

Method/path	Purpose
POST /api/receiving/grn	confirm/create GRN from PO receipt lines
GET /api/receiving/grn/{grn_id}	GRN detail
POST /api/returns	create return request
GET /api/returns/{return_id}	return detail

Gate, dock, receiving, and exit

The /api/gate-entries family provides:

- multipart gate-entry creation, list/detail, OCR scan, verification, pass download, and development reset;
- inbound arrival queue and dock list/create/update;
- dock assignment, movement, and check-in;
- unloading, received quantity, material condition, and quality checks;
- handling-unit generation and receiving completion;
- dock release, warehouse exit approval, and final gate exit;
- GRN drafts/posting, inventory transactions, and quantity-verification policy.

/api/gate exposes PO lookup and OCR/ANPR preview aliases. GET /api/dashboard/stats aggregates warehouse dashboard data.

Storage

/api/storage/putaway-tasks supports task listing, locations, inventory-location balances, handling-unit lookup, location assignment, start, and completion.

Procurement

All paths use /api/v1/procurement:

Family	Operations
/health, /stats, /global-search	health, metrics, cross-entity search
/vendor-types, /supplier-categories, /raw-materials	list/create reference data
/material-requests	next number, list/create/update/process
/material-stock	procurement inventory view
/suppliers	existence check, CRUD-like operations, block/unblock, upload documents
/rfqs	list/create/detail/send/decline/select supplier
/quotations	list/create/detail/update/reject/upload document
/purchase-orders	list/detail/by-number/PDF/approve/reject/resubmit/send
/finance-approvals	finance approval queue
/asns	next number, list/create/detail/update/attachment upload
/arrival-notifications, /notifications	list and read/read-all
/auth/supplier-login, /auth/change-password, /auth/dev-login	supplier and development authentication

8. Core workflows

```
Material request -> procurement process -> RFQ -> supplier quotation
-> supplier selection -> PO -> finance approve/reject/resubmit
-> PO sent -> supplier ASN -> arrival notification -> gate entry
```

```
Gate entry + evidence -> OCR/ANPR/manual verification -> inbound queue
-> dock assignment/check-in -> unload -> quantity/condition/quality checks
-> handling units -> receiving complete -> GRN/inventory transaction
-> putaway assignment/start/complete -> exit approval -> gate exit
```

Domain/application changes can persist an outbox record in the same database transaction. `relay_once` publishes pending events to Kafka and marks them published; the notification consumer persists relevant notification records. This prevents a committed business change from losing its event during temporary broker failure.

9. Data and migrations

The schema covers GRNs/receipt lines, returns, notifications/outbox, suppliers/contacts/documents, material requests/stock, RFQs/items, quotations, POs/approval history, ASNs/arrivals, gate entries/docks/receiving details, handling units, putaway tasks, and location balances.

```
cd D:\ams-wms-platform\backend\business-service
alembic upgrade head
alembic revision --autogenerate -m "describe change" # after model changes
```

Review generated migrations carefully: this history contains merged heads and legacy reconciliation. Scripts named `wipe_*` are destructive and require explicit approval, correct targeting, and a backup.

10. Authentication and authorization

Production intent is RS256 JWT validation against `JWT_JWKS_URL` with issuer and audience checks.

`get_current_user` maps claims to `CurrentUser`; authorization helpers enforce roles/permissions where endpoints attach them.

Development is intentionally permissive:

- `local`, `test`, and `development` override `get_current_user` with a local ADMIN/PROCUREMENT identity;
- procurement exposes a development login;
- the frontend can produce mock tokens.

These paths must not be enabled in production. Route authorization needs a production audit because not every current router operation declares an explicit role dependency.

11. Configuration

Pydantic Settings reads environment variables and `.env`. Use `.env.example` as the template.

Group	Variables
Runtime	<code>SERVICE_NAME</code> , <code>ENVIRONMENT</code> , <code>LOG_LEVEL</code> , <code>LOG_JSON</code>
Database	<code>DATABASE_URL</code> , pool size/overflow/echo
Auth	auth base URL, JWKS URL, issuer, audience, algorithm, cache TTL
Kafka/workers	bootstrap servers, client/group/topic/security, outbox interval/batch
HTTP/integration	<code>CORS_ALLOW_ORIGINS</code> , <code>REDIS_URL</code> , Gemini key/model
Observability	Prometheus and OTLP enablement/endpoint
Email	SMTP host/port/user/password/from name
Development	role-specific usernames/passwords

Never commit populated `.env` files, credentials, keys, or SMTP/Gemini secrets.

12. Local development

Full infrastructure:

```
cd D:\ams-wms-platform\backend
docker compose up -d
```

Service process:

```
cd D:\ams-wms-platform\backend\business-service
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
Copy-Item .env.example .env
alembic upgrade head
python -m uvicorn app.main:app --reload
```

Open <http://localhost:8000/docs>.

13. Tests and observability

```
pytest
ruff check app tests
```

Tests currently cover receiving/return domain logic, gate domain/services/API, and supplier identity. Add domain/use-case tests for rules and integration tests for persistence, auth, and transitions.

Request middleware propagates/generates correlation information. Exception handlers translate domain/application errors. Prometheus is enabled by default; OpenTelemetry requires deployment initialization. `/health/ready` does not currently probe PostgreSQL, Kafka, or JWKS, so production monitoring must check those separately plus outbox lag.

14. Extension rules

1. Keep rules/events framework-free in `domain`.
2. Put commands, use cases, and repository protocols in `application`.
3. Implement persistence and API adapters under `infrastructure`.
4. Register routers in `create_app()` and add an Alembic migration.
5. Add domain and integration tests; avoid new startup DDL.
6. Update this overview and let OpenAPI document field-level contracts.

15. Known risks and production checklist

- Startup mutates schema and suppresses several DDL errors; migrate only through Alembic.
- Gate/procurement routers are very large and mix concerns; split into use cases/services.
- Local auth and mock login are unsafe outside development; default-deny them.
- Readiness is not dependency-aware; add DB, Kafka, and auth/JWKS probes.
- Media is process-local; use durable object storage and file validation/scanning.
- Several skeleton modules and stale notes obscure actual status; keep the status table current.
- Redis/telemetry use is incomplete; configure or remove unused paths.
- Enforce least privilege on every sensitive endpoint, TLS/restrictive CORS/Kafka security, backups, outbox alerts, and CI tests before production.

Part II — Frontend Application

1. Scope

The frontend directory contains the NexusWMS browser application for warehouse, gate-security, procurement, supplier, and finance users. It provides dashboards and guided workflows over the FastAPI business service.

Routes under `src/routes` define screens; `src/lib/api-client.ts` is the central backend integration.

2. Stack

Concern	Technology
UI/runtime	React 19, TypeScript 5.8, TanStack Start
Routing/data	TanStack Router file routes, TanStack Query 5
Server/build	Vite 8, Nitro
Styling/components	Tailwind CSS 4, Radix UI, local primitives
Supporting UI	Lucide, Sonner, Recharts
Codes/OCR	QRCode, JsBarcode, Tesseract.js
Quality	ESLint, typescript-eslint, Prettier

3. Runtime and source architecture

```

Browser -> TanStack route -> AppShell/page -> central fetch API client -> :8000
| | |
| | +-> component state/effects
| +-> QueryClient context
+-> localStorage: auth_token, user_info, selected drafts/workflow data

```

```

src/
├─ routes/ file-based pages/nested layouts
├─ components/ui/ reusable Radix/custom primitives
├─ components/wms/ AppShell and domain components
├─ lib/api-client.ts backend HTTP operations
├─ lib/auth-utils.ts auth and role guards
├─ lib/wms-data.ts static/local domain data
├─ router.tsx Router + QueryClient
├─ routes/__root.tsx HTML shell, providers, global guards/errors
├─ routeTree.gen.ts generated; never hand-edit
├─ server.ts / start.ts TanStack Start/Nitro entries
└─ styles.css Tailwind/theme/global styles

```

`router.tsx` builds the QueryClient and Router. `__root.tsx` supplies metadata, CSS, auth redirect, Query provider, outlet, toaster, and root error/not-found UI. Most authenticated pages use `components/wms/app-shell.tsx` for role navigation, user identity, notifications, search, and logout.

4. Roles and navigation

Role	Home and navigation
WAREHOUSE / operators	<code>/warehouse-dashboard</code> : inventory, putaway, requests, arrivals, exit, docks, receiving, reports
PROCUREMENT	<code>/procurement-dashboard</code> : suppliers, requests, RFQs, quotations, POs, ASNs
SUPPLIER	<code>/supplier-dashboard</code> : quotations and ASN creation
FINANCE	<code>/finance-dashboard</code> : pending approvals and reports
GATE_SECURITY	<code>/gate-dashboard</code> : gate entry, arrivals, exit

Role	Home and navigation
ADMIN	general/fallback behavior

`requireRole` redirects users without a required role to their role home. Client guards control UX only; the backend must enforce authorization.

5. Route catalog

Shared

Route	Purpose
/login, /	login and role-aware entry redirect
/notifications	role and arrival notifications
/settings, /reports	settings and reporting surfaces

Gate and warehouse

Route	Purpose
/gate-dashboard, /gate-entry	gate metrics and entry capture
/driver-verification, /vehicle-verification	verification steps
/accept-arrival, /arrival-success, /dock-assignment	arrival workflow steps
/warehouse-dashboard	warehouse dashboard
/vehicle-queue, /dock-management	inbound queue and dock administration
/receiving, /grn	unload/check/complete receiving and post GRN
/inventory, /putaway-tasks	stock/location views and putaway execution
/vehicle-exit	warehouse approval and gate exit
/warehouse/material-requests	create/edit material requests

Procurement, supplier, and finance

Route	Purpose
/procurement-dashboard	metrics, alerts, supplier/PO search
/master-data, /new-supplier, /supplier/\$supplierId	supplier list/onboarding/detail
/procurement/material-requests, /procurement/new-rfq, /procurement/rfqs	sourcing inputs and RFQs
/procurement/quotations	compare/evaluate/select quotations
/procurement/purchase-orders, /purchase-order	PO list/detail/PDF/send
/procurement/asns, /procurement/asns/\$asnId	ASN list/detail
/supplier-dashboard, /submit-quotation, /supplier/asns/new	supplier portal workflows
/finance-dashboard, /finance/approvals	finance overview and approval list
/finance/approvals/\$approvalId, /finance/approvals/compare/\$rfqId	approval detail/comparison

6. Backend integration

The native-fetch client:

- uses `http://<browser-hostname>:8000`, or a hard-coded LocalTunnel backend for `loca.lt`;

- attaches `auth_token` as a Bearer token (or a local mock token when absent);
- sets JSON content type, parses JSON, and normalizes FastAPI validation errors;
- has specialized multipart upload and PDF/blob download methods.

It covers gate/dock/receiving/exit, storage/putaway, GRN/returns, suppliers/reference data, material requests/stock, RFQs, quotations, POs/approvals, ASNs, notifications, and search.

No general `VITE_*` API base exists. Production must preserve the host/port convention or move this to `environment/reverse-proxy` configuration.

7. Authentication

Login currently follows:

1. `supplier_*` usernames call supplier login.
2. Other names call the business-service development login.
3. Failure creates a mock role/token based on username text.

`auth_token` and `{token, username, roles, supplierId?, mustChangePassword?}` in `user_info` are stored in `localStorage`. The root redirects unauthenticated users to `/login`; pages apply `requireAuth/requireRole`; logout removes both keys.

Mock login is development-only. `localStorage` tokens are XSS-accessible, browser guards are not authorization, and production needs the real auth-service login/refresh/logout flow.

8. State, UI, and errors

TanStack Query is globally available, but current pages primarily load through effects and local state, so loading/error/refetch logic is page-specific. `localStorage` also persists ASN drafts; router search/navigation state links some gate steps. `wms-data.ts` and route constants provide local/static data where backend coverage is incomplete.

New server state should use typed TanStack Query hooks, stable keys, invalidation, and cancellation. Reduce any at API boundaries with generated/shared OpenAPI types.

UI primitives under `components/ui` wrap Radix behavior and Tailwind classes. `components/wms/primitives.tsx` adds domain presentation. `AppShell` supplies desktop sidebar/header and mobile bottom navigation. The root has error and 404 boundaries; API errors become `JavaScript Errors` and routes generally display inline feedback or Sonner toasts.

Camera/OCR/QR/barcode functions require real device permission and responsive testing. Supplier onboarding directly calls an external postal PIN-code service, creating availability and privacy dependencies.

9. Development and verification

```
cd D:\ams-wms-platform\frontend
npm install
npm run dev # http://localhost:8080
npm run build # production build
npm run build:dev
npm run preview
npm run lint
npm run format # writes formatting changes
```

The backend is expected on port 8000. Never edit `src/routeTree.gen.ts`; the router plugin generates it.

There is no automated frontend test suite today. Minimum verification is:

```
npm run lint
npm run build
```

Manually test login/logout, role redirects, changed API failures, direct refresh, desktop/mobile layout, and camera/file permissions. Recommended additions: Vitest/Testing Library, MSW contract tests, and Playwright for procurement-to-arrival and gate-to-putaway journeys.

10. Adding a feature

1. Add a typed method/DTO in `api-client.ts`.
2. Add the file route and appropriate `beforeLoad` role guard.
3. Reuse UI/WMS primitives and use TanStack Query for reusable server state.
4. Add the role navigation item in `app-shell.tsx` if required.
5. Cover loading, empty, error, permission, and mobile states.
6. Run lint/build and add tests; do not edit the generated route tree.

11. Known risks and production checklist

- API URL and LocalTunnel behavior are embedded in source; make the base deploy-time configurable.
- Remove development login, mock token fallback, and default mock authorization header.
- Replace API and page `any` types with OpenAPI-generated/shared contracts.
- Consolidate server state on TanStack Query and split large pages/API client by domain.
- Add automated unit, contract, accessibility, and end-to-end tests in CI.
- Replace local/static placeholders where production data is expected.
- Implement a secure token/session lifecycle and CSP/XSS hardening.
- Define upload limits/types, camera/OCR consent, and external-service privacy behavior.
- Audit all route guards, responsive states, and error handling; never log tokens, documents, credentials, or personal data.